CONCAT in MySQL

Strong Definition

CONCAT() in MySQL is a **string function** used to combine two or more values into a single string.

It joins:

- text + text
- text + number
- column + text
- multiple columns

---

1. Beginner Explanation

Suppose you have:

first_name	last_name
John	Doe

You want:

John Doe

Use:

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM employees;
```

Output:

full_name
John Doe

---

2. Syntax

CONCAT(value1, value2, value3, ...)

It joins everything in order.

---

3. Deep Explanation (How It Works Internally)

MySQL processes CONCAT() row by row.

If table has 1 million rows:

```
SELECT CONCAT(first_name,' ',last_name)
FROM users;
```

MySQL does:

For each row:

- read first_name
- read space ' '
- read last_name
- combine into new temporary string
- return result

Query Execution Order Example

```
SELECT CONCAT(first_name,last_name)
FROM employees
WHERE dept = 'IT';
```

Logical execution:

1. FROM employees
2. WHERE dept='IT'
3. SELECT CONCAT(...)

So filtering happens first.

Good for performance.

---

4. Real-Time Use Cases

A. Full Name Display

```
SELECT CONCAT(first_name,' ',last_name) AS full_name
FROM users;
```

Used in admin dashboards.

B. Employee Code

```
SELECT CONCAT('EMP-', emp_id)
FROM employees;
```

Output:

EMP-101

C. Address Formatting

```
SELECT CONCAT(city, ', ', state, ', ', country)
FROM customers;
```

D. Product Labels

```
SELECT CONCAT(product_name,' - ₹',price)
FROM products;
```

E. Reports / Export Files

```
SELECT CONCAT(order_id,' | ',customer_name)
FROM orders;
```

---

5. Important Behavior in MySQL

Numbers auto-convert to string

```
SELECT CONCAT('ID-',101);
```

Output:

ID-101

---

6. VERY IMPORTANT: NULL Behavior

In MySQL:

```
SELECT CONCAT('John', NULL, 'Doe');
```

Output:

NULL

If **any argument is NULL**, result becomes NULL.

Safer Version

Use IFNULL() or COALESCE():

```
SELECT CONCAT('John', IFNULL(NULL,''), 'Doe');
```

Output:

JohnDoe

---

7. CONCAT_WS() (Better in Many Cases)

Means:

Concatenate With Separator

SELECT CONCAT_WS(' ', first_name, last_name);

Output:

John Doe

Separator added automatically.

Better than:

CONCAT(first_name,' ',last_name)

---

8. Common Mistakes

Mistake 1: No Spaces

SELECT CONCAT('John','Doe');

Output:

JohnDoe

Mistake 2: NULL Surprise

SELECT CONCAT(name,NULL);

Returns NULL.

Mistake 3: Using in WHERE

Bad:

WHERE CONCAT(first_name,last_name)='JohnDoe'

May prevent index use.

Better:

WHERE first_name='John'

AND last_name='Doe'

Mistake 4: Thinking It Adds Numbers

SELECT CONCAT(10,20);

Output:

1020

Not:

30

---

9. Performance Awareness

Good Use:

SELECT CONCAT(...)

Usually fine.

Slower Use:

WHERE CONCAT(...)

JOIN ON CONCAT(...)

ORDER BY CONCAT(...)

Because MySQL computes each row.

---

10. Real Company Example

E-commerce

SELECT CONCAT(product_name,' - ₹',price)

FROM products;

Website product labels.

HR System

SELECT CONCAT(first_name,' ',last_name) AS employee_name

FROM employees;

Banking

SELECT CONCAT('ACC-',account_no)

FROM accounts;

---

13. Pro Tip

Use CONCAT_WS() when separator needed.

Use COALESCE() when NULL possible.

---

14. Interview Note

Difference:

CONCAT()

joins values.

GROUP_CONCAT()

joins multiple row values into one row.

CONCAT_WS in MySQL

Strong Definition

CONCAT_WS() in MySQL means:

Concatenate With Separator

It joins multiple strings into one string **using a separator automatically placed between values**.

WS = With Separator

---

1. Beginner Explanation

Instead of writing:

SELECT CONCAT(first_name,' ',last_name);

You can write:

SELECT CONCAT_WS(' ', first_name, last_name);

Output:

John Doe

Cleaner and better.

---

2. Syntax

CONCAT_WS(separator, value1, value2, value3, ...)

First argument = separator.

Remaining arguments = values to join.

---

3. Simple Examples

Example 1: Space Between Names

SELECT CONCAT_WS(' ', 'John', 'Doe');

Output:

John Doe

Example 2: Comma Separator

SELECT CONCAT_WS(', ', 'Bangalore', 'Karnataka', 'India');

Output:

Bangalore, Karnataka, India

Example 3: Hyphen

SELECT CONCAT_WS('-', 'EMP', 101);

Output:

EMP-101

---

4. Deep Explanation (How It Works Internally)

MySQL processes row by row.

For each row:

CONCAT_WS(' ', first_name, last_name)

Steps:

1. Read separator ' '
2. Read first_name

3. Read last_name
4. Place separator between non-null values
5. Return final string

---

5. Key Difference vs CONCAT()

CONCAT

CONCAT(first_name,' ',last_name)

You manually place separators.

CONCAT_WS

CONCAT_WS(' ',first_name,last_name)

Separator auto-added.

Much cleaner for many columns.

---

6. VERY IMPORTANT: NULL Handling

This is where CONCAT_WS() shines.

Example:

SELECT CONCAT_WS(' ', 'John', NULL, 'Doe');

Output:

John Doe

It **ignores NULL values**.

Compare with CONCAT()

SELECT CONCAT('John', NULL, 'Doe');

Output:

NULL

Huge difference.

---

7. Real-Time Use Cases

A. Full Name

```
SELECT CONCAT_WS(' ', first_name, middle_name, last_name)
FROM users;
```

If middle_name NULL → still clean output.

B. Address Formatting

```
SELECT CONCAT_WS(', ', city, state, country)
FROM customers;
```

Output:

Mumbai, Maharashtra, India

C. Product Labels

```
SELECT CONCAT_WS(' - ', product_name, category, brand)
FROM products;
```

D. Employee IDs

```
SELECT CONCAT_WS('-', 'EMP', emp_id, dept_id)
FROM employees;
```

---

8. Why Companies Prefer CONCAT_WS()

Because real data has NULL values.

Example:

first_name	middle_name	last_name
John	NULL	Doe

Using CONCAT:

NULL

Using CONCAT_WS:

John Doe

Cleaner reports.

---

9. Common Mistakes

Mistake 1: Separator Must Be First

Wrong:

CONCAT_WS(first_name,' ',last_name)

Correct:

CONCAT_WS(' ',first_name,last_name)

Mistake 2: Empty String vs NULL

CONCAT_WS(' ', 'John', '', 'Doe')

Output may include extra spacing because empty string is not NULL.

Mistake 3: Using in WHERE

Bad:

WHERE CONCAT_WS(' ',first_name,last_name)='John Doe'

May hurt indexes.

Better:

WHERE first_name='John'

AND last_name='Doe'

---

10. Performance Awareness

Good:

Use in SELECT output.

SELECT CONCAT_WS(...)

Avoid Heavy Use In:

- WHERE
- JOIN conditions
- ORDER BY large datasets

---

11. Execution Order Example

SELECT CONCAT_WS(' ',first_name,last_name)

FROM employees

WHERE dept='IT';

Logical order:

1. FROM
2. WHERE
3. SELECT (CONCAT_WS() runs here)

---

14. Interview Summary

Use CONCAT when:

- exact control needed

Use CONCAT_WS when:

- separator needed
- multiple columns
- NULL values possible

Golden Rule

For names, addresses, labels → **prefer CONCAT_WS()**

SUBSTRING in MySQL

Strong Definition

SUBSTRING() in MySQL is a **string function** used to extract part of a string starting from a specific position, optionally for a specific length.

Used when you need:

- first few characters
- last few characters
- part of IDs
- domain from email
- masked values

- parsing codes

---

1. Beginner Explanation

If text is:

OpenAI

You want only:

Open

Use:

SELECT SUBSTRING('OpenAI',1,4);

Output:

Open

---

2. Syntax

Standard Form

SUBSTRING(string, start, length)

- string = text
- start = starting position
- length = how many characters to take

Without Length

SUBSTRING(string, start)

Returns from start till end.

---

3. Position Rules (Very Important)

MySQL starts counting from **1**, not 0.

Text	Position
H	1
e	2
l	3
l	4

Text Position

o 5

---

4. Examples

Example 1: First 3 Characters

SELECT SUBSTRING('Hello',1,3);

Output:

Hel

Example 2: From Position 3 Till End

SELECT SUBSTRING('Hello',3);

Output:

llo

Example 3: Middle Part

SELECT SUBSTRING('Database',5,4);

Output:

base

---

5. Deep Explanation (How It Works Internally)

MySQL receives:

SUBSTRING(name,2,3)

For each row:

1. Read name
2. Find 2nd character
3. Extract next 3 chars
4. Return result

Runs row by row.

---

6. Query Execution Order Example

```
SELECT SUBSTRING(first_name,1,3)
FROM employees
WHERE dept='IT';
```

Logical order:

1. FROM
2. WHERE
3. SELECT (SUBSTRING() executes here)

Rows filtered first.

---

7. Real-Time Use Cases

A. Mask Phone Numbers

```
SELECT SUBSTRING(phone,1,3)
FROM customers;
```

B. Extract Email Prefix

```
SELECT SUBSTRING(email,1,5)
FROM users;
```

C. Show Last 4 Digits Card

```
SELECT SUBSTRING(card_no,-4);
```

Output:

1234

D. Parse Employee Code

EMP2025IT01

Extract year:

```
SELECT SUBSTRING(code,4,4);
```

E. Product Prefixes

```
SELECT SUBSTRING(product_id,1,3)
```

FROM products;

---

8. VERY IMPORTANT: Negative Positions

MySQL allows negative start positions.

Example:

SELECT SUBSTRING('OpenAI',-2);

Output:

AI

Starts from end.

Example:

SELECT SUBSTRING('Database',-4,2);

Output:

ba

---

9. Common Mistakes

Mistake 1: Using 0 Position

SUBSTRING('Hello',0,2)

Unexpected / empty depending behavior.

Use position 1+.

Mistake 2: Confusing with Python Indexing

Python starts at 0. MySQL starts at 1.

Mistake 3: Wrong Length

SUBSTRING('Hello',2,10)

Returns available chars only.

Output:

ello

Mistake 4: Using in WHERE on Indexed Columns

Bad:

WHERE SUBSTRING(phone,1,3)='987'

May avoid index usage.

Better:

WHERE phone LIKE '987%'

---

10. Performance Awareness

Good Use:

SELECT SUBSTRING(name,1,3)

Output formatting.

Slower Use:

WHERE SUBSTRING(col,...)

JOIN ON SUBSTRING(...)

ORDER BY SUBSTRING(...)

Because function computed row by row.

---

11. Real Company Use Cases

Banking

Show masked account:

XXXXXX1234

Using last 4 digits.

E-commerce

Extract SKU prefix.

HR

Generate initials:

JOH

Telecom

Filter mobile circle code prefixes.

---

14. Interview Notes

Difference:

LEFT(str,n)

Gets first n chars.

RIGHT(str,n)

Gets last n chars.

SUBSTRING()

Flexible start + length.

---

15. Golden Rule

Use SUBSTRING() for extraction, not filtering indexed data when better alternatives exist.

REPLACE in MySQL

Strong Definition

REPLACE() in MySQL is a **string function** used to find occurrences of a substring inside a string and replace them with another substring.

It does **text substitution**, not row replacement.

---

1. Beginner Explanation

If you have:

Hello World

and want:

Hello MySQL

Use:

SELECT REPLACE('Hello World', 'World', 'MySQL');

Output:

Hello MySQL

---

2. Syntax

REPLACE(original_string, from_string, to_string)

- original_string = full text
- from_string = what to search for
- to_string = replacement text

---

3. Deep Explanation (How It Works Internally)

For each row, MySQL:

1. Reads the string
2. Searches for matching substring occurrences
3. Replaces all matches
4. Returns new string

Example:

SELECT REPLACE('banana', 'a', 'o');

Process:

- banana
- replace all a with o

Output:

bonono

---

4. Query Execution Order Example

SELECT REPLACE(name,'Mr. ','')

FROM students

WHERE dept='IT';

Logical order:

1. FROM
2. WHERE
3. SELECT (REPLACE() runs here)

Rows filtered first.

---

5. Real-Time Use Cases

A. Clean Phone Numbers

```
SELECT REPLACE(phone,'-','')
FROM customers;
```

Convert:

987-654-3210

to

9876543210

B. Remove Prefix

```
SELECT REPLACE(name,'Mr. ','')
FROM users;
```

C. Update URLs

```
SELECT REPLACE(url,'http://','https://')
FROM websites;
```

D. Currency Symbol Cleanup

```
SELECT REPLACE(price,'₹','')
FROM products;
```

E. Standardize Spaces

```
SELECT REPLACE(address,'  ',' ')
FROM customers;
```

---

6. Important Behavior

Replaces ALL Matches

```
SELECT REPLACE('aaaa','a','b');
```

Output:

bbbb

Not just first match.

Case Sensitivity

Depends on collation.

Often MySQL string comparisons follow column/database collation rules.

---

7. Common Mistakes

Mistake 1: Confusing with SQL REPLACE Statement

MySQL also has a **REPLACE statement** (for rows).

This topic is function:

REPLACE()

Not:

REPLACE INTO table_name ...

Different concept.

Mistake 2: Expecting Original Data to Change

SELECT REPLACE(name,'a','x')

Only changes output.

To permanently change data:

UPDATE students

SET name = REPLACE(name,'a','x');

Mistake 3: Wrong Search Text

REPLACE('Hello','he','X')

May not match if case-sensitive collation differs.

Mistake 4: NULL Input

SELECT REPLACE(NULL,'a','b');

Returns:

NULL

---

8. Performance Awareness

Good Use:

SELECT REPLACE(col,...)

Formatting output.

Slower Use:

WHERE REPLACE(phone,'-','')='9876543210'

Because function runs row by row.

Better store clean data or use indexed normalized column.

---

9. Real Company Use Cases

Data Cleaning Pipelines

Remove symbols/spaces before analytics.

CRM Systems

Standardize phone numbers.

E-commerce

Fix imported product names.

Banking

Mask account formatting characters.

---

12. Interview Notes

Difference:

REPLACE()

Replace text inside string.

UPDATE ... SET

Changes stored data.

REPLACE INTO

Insert/overwrite rows.

---

13. Golden Rule

Use REPLACE() for string cleanup and formatting, not heavy filtering on indexed columns.

---

14. Bonus Example with Your Students Table

```
SELECT roll_no,
    REPLACE(student_name,'a','@') AS styled_name
FROM students;
```

String Functions — LOWER, UPPER, REVERSE

These are string functions used to change text format.

---

1. LOWER()

Definition:

Converts all letters into **small letters** (lowercase).

Example:

```
SELECT LOWER('INDRA');
```

Output:

indra

Real Use:

- emails to lowercase
- usernames standard format
- case-insensitive comparisons

Example with table:

```
SELECT LOWER(student_name)
FROM students;
```

---

2. UPPER()

Definition:

Converts all letters into **capital letters** (uppercase).

Example:

SELECT UPPER('indra');

Output:

INDRA

Real Use:

- display names in caps
- codes
- standard reports

Example:

SELECT UPPER(student_name)

FROM students;

---

3. REVERSE()

Definition:

Reverses the text from end to start.

Example:

SELECT REVERSE('INDRA');

Output:

ARDNI

Real Use:

- fun checks
- palindrome logic
- reverse pattern testing
- masking logic sometimes

Combined Example

SELECT

LOWER('InDrA'),

UPPER('InDrA'),

REVERSE('InDrA');

Output:

Result

indra

INDRA

ArDnI

Students Table Example

SELECT

student_name,

LOWER(student_name),

UPPER(student_name),

REVERSE(student_name)

FROM students;

Easy Memory Trick

- LOWER() → lower case
- UPPER() → upper case
- REVERSE() → opposite direction

Performance Note

Good in SELECT.

Avoid in WHERE on large indexed columns like:

WHERE LOWER(name)='indra'

Better store normalized data.

Golden Rule

These functions **change display/output**, unless used in UPDATE.

Golden Rule Explained

Statement:

These functions change display/output, unless used in UPDATE.

Meaning:

Functions like:

- LOWER()
- UPPER()
- REVERSE()

usually change **what you see in query result**, not the actual stored data.

---

1. Example — Only Display Changes

Table:

id	name
1	Indra

Query:

```
SELECT UPPER(name)
FROM students;
```

Output:

INDRA

But actual table data is still:

id	name
1	Indra

Because SELECT only shows result.

It does **not** modify stored data.

Think Like This

SELECT

= Ask database to show data.

Not change data.

---

2. Example — Real Data Changes with UPDATE

UPDATE students

SET name = UPPER(name);

Now MySQL:

- reads each row
- converts name to uppercase
- saves back into table

Now actual stored data becomes:

id	name
1	INDRA

Why This Matters

Many beginners think:

SELECT LOWER(name)

changes database permanently.

It does **not**.

It only changes output temporarily.

Real World Example

Login Emails

Display lowercase:

SELECT LOWER(email)

FROM users;

No permanent change.

Data Cleaning Project

Need permanent standardization:

UPDATE users

SET email = LOWER(email);

Now stored emails become lowercase.

Interview Style Summary

SELECT + Function

SELECT UPPER(name)

✓ output changes only

UPDATE + Function

UPDATE table

SET name = UPPER(name)

✓ stored data changes

Easy Analogy

SELECT

Like applying a filter to photo preview.

UPDATE

Like saving edited photo permanently.

Golden Rule (Remember Forever)

If function is inside:

SELECT

Temporary result.

UPDATE SET column = function(...)

Permanent stored change.